

Framework for System Simulation

Thesis – Specification

Florian Frech, 11.08.2025

1. Requirements Engineering

1. Requirements Engineering – Theor
2. Methodology
3. Functional Requirements
 1. Modeling
 2. Heterogeneity & Simulation
 3. User Interaction
4. Non-Functional Requirements

2. Existing Framework – SysSimX

1. Conceptual Overview
2. Framework Classes: Attributes and Methods
3. **Simulation Flow**
 1. Creation of Module Objects
 2. Creation of Simulation Objects
 3. Connection Modules
 4. Simulation Execution
 5. Postprocessing

3. Framework Evaluation

1. Modeling
2. Heterogeneity & Simulation
3. User Interaction

1. Summary
2. Potential Improvements

4. Alternative Approach for System Simulation Framework

5. Further Procedure

Purpose:

- Requirements for a system are the **description of what the system should do** (services that it provide and constraints on its operation), reflect the needs of customers for a system
- **Requirements Engineering (RE): process of**
 - finding out
 - analyzing
 - documenting
 - checking these services and constraints
- **User Requirements:** High-level abstract requirements, statements in a natural language & diagrams of what services the system is expected to provide to system users
- **System Requirements:** more detailed descriptions of the software system's functions, services, and operational constraints
- A user requirement (quite general requirement) is expanded into several system requirements

Functional Requirements:

- Statements of **services the system should provide**, how the system should react to particular inputs, how the system should behave in particular situations, and may also explicitly state what the system should not do
- Depend on type of software being developed and the expected users of the software
- As functional user requirements: described in an abstract way that can be understood by system users
- As functional system requirements: describe the system functions in detail
- **Completeness:** all services required by the users should be defined
- **Consistency:** requirements should not have contradictory definitions
- In practice **impossible to achieve requirement consistency and completeness**
 - Easy to make mistakes and omissions when writing specifications for complex systems
 - Many stakeholders with different often inconsistent needs (not obvious in the first specification, emerge after deeper analysis)

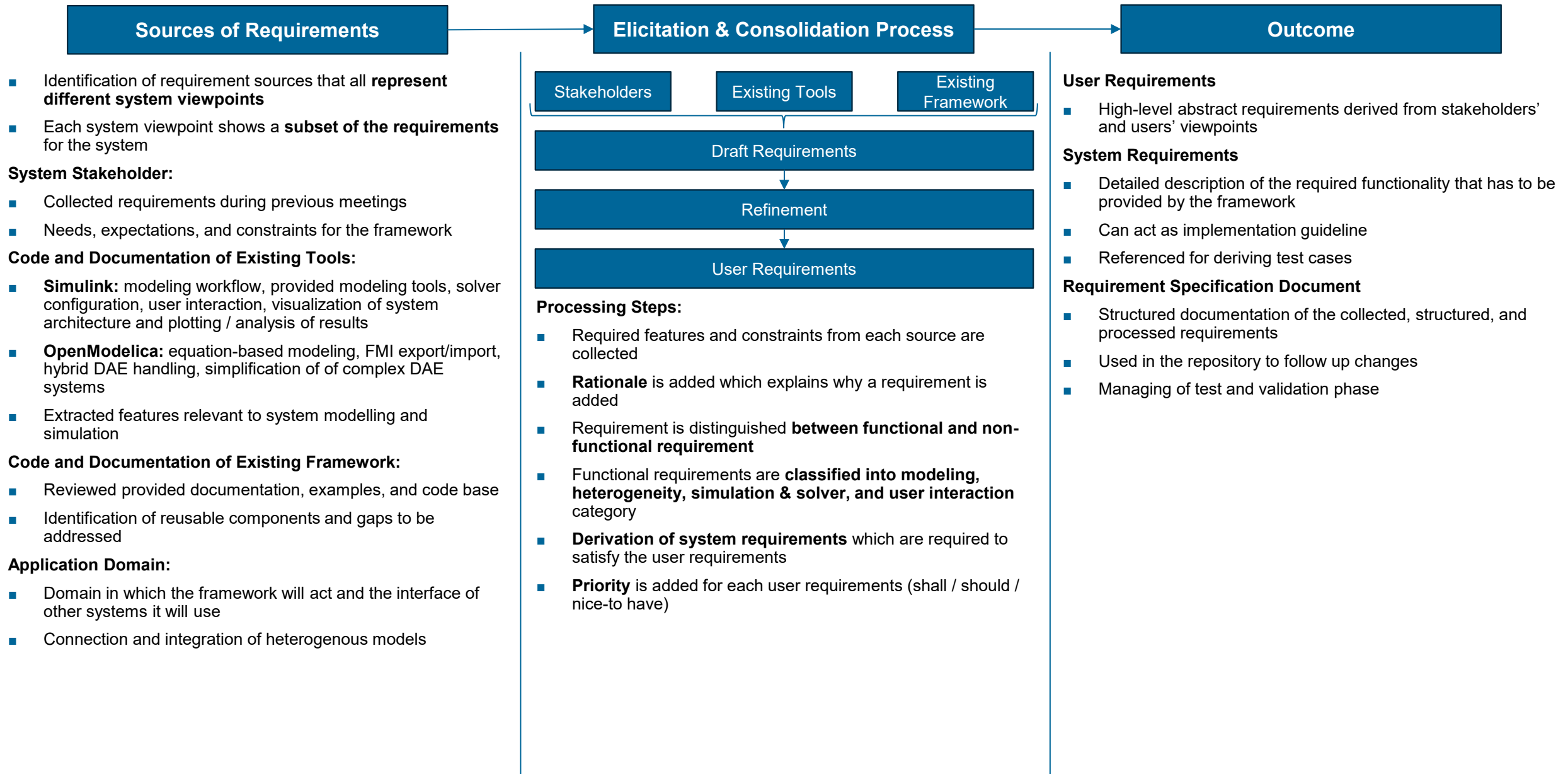
Non-Functional Requirements:

- Requirements that are **not directly concerned with the specific services** delivered by the system to the users
- Specify or constrain characteristics of the system as a whole (performance, security, availability)
- Are often more critical than individual functional requirements
- Difficult to relate software components to non-functional requirements
 - May **affect the overall architecture** of a system rather than the individual components
 - May **generate several related functional requirements** that define new system services that are required

Process activities of requirement elicitation and analysis:

No	Name	Description
1	Requirement Discovery	• interacting with stakeholders of the system to discover the requirements • domain requirements from stakeholders and documentation are discovered during this activity
2	Requirement Classification and Organization	• takes the unstructured collection of requirements, groups related requirements, and organizes them into clusters • Use a model of the system architecture to identify sub-systems and to associate requirements with each sub-system • Practice: requirements engineering and architectural design cannot be separate activities
3	Requirement Prioritization and negotiation	• usually, requirements will conflict • activity is concerned with prioritizing requirements and finding and resolving requirements conflicts through negotiation • stakeholders must meet to resolve differences and agree on compromise requirements
4	Requirements Specification	• Requirements are documented and input into the next round of the spiral • Formal or informal requirements documents may be produced

(Sommerville, 2011)



ID	User Requirement	Rationale	Derived System Requirements	Category
UR-1	The framework shall support the component-based modeling of multi-domain systems .	- Supporting multiple physical domains (e.g., electrical, mechanical, control) - simulating realistic complex mechatronic systems	- Define and instantiate reusable component models - Provide a unified interface for connecting components across different physical domains.	Shall
UR-2	The framework shall support the use of units for all physical quantities .	avoid modeling errors, supports consistency checks - improves the clarity and correctness of models.	associate a unit with every model parameter, variable, input, and output - validate unit compatibility when connecting components	Shall
UR-3	The framework shall allow the creation of system models by connecting subsystems .	- enables hierarchical modeling - reflects the structure of the systems. - enables reuse / simplifies by isolating functionality	- support hierarchical composition of system models from individual subsystems - provide standardized interface variables for defining subsystem inputs and outputs.	Shall
UR-4	The framework should allow the acausal equation-based modeling of the component behavior .	- Describe component behavior based on physical laws without prescribing the computational direction - enables flexible model reuse and better abstraction multi-domain physical systems	- Support bidirectional connectors to exchange variables between components - identify and classify equations as differential, algebraic, or event-related. - Computational causality analysis, Index analysis and apply index reduction methods (e.g., Pantelides algorithm) when required	Should
UR-5	The framework shall enable the causal equation-based modeling of the component behavior .	- Enforces a defined computational direction (explicit inputs and outputs) simplifies simulation execution and debugging - particularly for signal-processing or control-oriented systems where causality is naturally defined	- Allow the definition of components with explicitly declared input and output - Transformation of causal equations into executable functions - shall integrate with ODE solvers for forward simulation. - solve algebraic loops that arise from strongly connected components.	Shall
UR-6	The framework shall support the definition of component behavior using discrete-time or hybrid dynamics .	Systems (e.g., embedded controllers, digital electronics, switching systems) exhibit behavior that involves discrete events or combines continuous and discrete-time dynamics (hybrid systems).	- support modeling elements with discrete-time update rules (e.g., difference equations, sampling intervals). support the reinitialization of continuous states in response to discrete events or switching conditions.	Shall
UR-7	The framework should provide templates for the creation of equation-based component models .	- Reusable templates simplify the modeling process Ensures consistency across models	- should define a library of reusable templates for common physical components - Ensure that templates support parameterization and unit annotation.	Shall
UR-8	The framework shall allow the reuse of components and subsystems with different configurations.	- Enhances modeling efficiency, reduces redundancy, and promotes modularity Different configurations allows the same component or subsystem to be adapted for various simulation contexts without reimplementa	Support parameterized instantiation of components and subsystems - Allow multiple instances of the same model template with independent settings.	Shall

ID	User Requirement	Rationale	Derived System Requirements	Category
UR-9	The framework shall enable the integration of subsystem models developed using different modeling tools.	- CPSs often comprise parts designed with domain-specific tools (e.g., OpenModelica OpenSim, Simulink, FEM) - framework must support tool interoperability to allow co-simulation or model exchange	- Support importing and interfacing with models via standardized interfaces such as FMI (Functional Mock-up Interface) - Provide adapter components or wrappers to interface models from tools without native FMI support - Allow users to configure tool-specific parameters and settings required for model initialization and execution.	Shall
UR-10	The framework should support the combination of equation-based and other heterogeneous sub models.	- Some subsystems are best described with acausal equations (e.g., physical domains) - Others may come from black-box models, real-time control systems, or specialized tools - Combining these allows for flexible system modeling	- Provide a unified interface abstraction for coupling equation-based and non-equation-based subsystem models support data exchange mechanisms that are compatible with both - should manage causality alignment between acausal and causal sub models	Should

ID	User Requirement	Rationale	Derived System Requirements	Category
UR-11	The framework shall enable the numerical simulation of hybrid systems with both continuous and discrete-time behavior.	- Supporting hybrid systems is essential for simulating real-world mechatronic or cyber-physical systems accurately - Arise when controller logic sub system (discrete events) is coupled to continuous sub systems (continuous states)	- The system shall support a simulation loop capable of handling continuous integration and discrete-time updates within a compatible time axis. - The system shall allow users to define the sample times for discrete-time components independently from the integration step size. - The system shall maintain event handling mechanisms for discrete transitions such as switches, zero-crossings, or state changes	Shall
UR-12	The framework shall allow the user to configure and select solver types and step sizes for simulation.	Different system dynamics and simulation goals require different numerical solvers - e.g., explicit vs. implicit, fixed vs. adaptive step size	- Provide an interface for selecting from multiple solver types (e.g., fixed-step, variable-step, stiff solvers) - Allow user-defined step sizes for fixed-step solvers and tolerance settings for variable-step solvers. Shall validate solver compatibility with the modeled system (e.g., reject explicit solvers for stiff systems with algebraic loops)	Shall

ID	User Requirement	Rationale	Derived System Requirements	Category
UR-13	The framework should provide a visual representation of the system model and its hierarchical structure .	- Visualizing systems and their composition helps users understand model structure, debug connectivity issues, and communicate the design more effectively. - Hierarchical views support scalability and modular design	- The system should generate directed graphs representing components and their interconnections - The system should visualize the hierarchical composition of subsystems through expandable/collapsible views. - The system should allow exporting the visual representation to a file (e.g., PNG, SVG, PDF)	Should
UR-14	The framework should allow the user to observe simulation progress through graphical and textual output.	- Monitoring simulation progress provides immediate feedback - It helps to detect runtime issues (e.g., divergence, discontinuities), and enhances user confidence.	- The system should provide real-time plotting of selected output variables during simulation. - Should log simulation progress with time-stamped messages including solver status, time step, and convergence information. - Allow users to configure which signals or outputs are monitored during simulation.	Should
UR-15	The framework should allow the user to compare simulation results across different configurations .	Comparing simulations with different model parameters, solvers, or subsystem configurations - Essential for sensitivity analysis, performance evaluation, optimization, and design decisions.	- The system should allow users to define and execute multiple simulation configurations - The system should store simulation results in a structured format with metadata describing the configuration used - The system should provide tools to visualize and compare selected output signals across configurations.	Should
UR-16	The framework should support result export and post-processing functionality.	- Exporting and analyzing simulation results outside the framework is often necessary for documentation, reporting, optimization, or further processing in external tools (e.g., Excel, Python, MATLAB) - Post-processing enhances the usability of results and facilitates integration into engineering workflows	- The system should provide export functionality for simulation results in common data formats (e.g., CSV, Excel, JSON, HDF5) - The system should support time series filtering, resampling, and statistical post-processing of simulation outputs. - The system should associate exported results with metadata such as model configuration, simulation settings, and timestamps.	Should

ID	User Requirement	Rationale	Derived System Requirements	Category
NFR-1	The framework should provide clear error messages and logs .	• Clear error messages and informative logs • Help users to quickly identify and resolve issues during model creation, simulation setup, and execution.	• Logging mechanism that captures events, warnings, and errors at multiple verbosity levels (e.g., DEBUG, INFO, WARNING, ERROR) • Provide contextual error messages that include the source module, variable, and cause of failure	Shall
NFR-2	Documentation and examples should help users to build and simulate models.	• Improve usability and reduce the learning curve for new users and developers • Encouraging adoption and correct usage of the framework.	• Provide structured documentation that includes component creation, system assembly, simulation configuration, and error handling • Include a set of example models of varying complexity, demonstrating key features such as hybrid dynamics, hierarchical composition, and tool integration	Shall
NFR-3	System must work on standard developer environment (Python = 3.8 on Linux/Windows required for OpenSim)	• Ensuring compatibility with commonly used platforms and Python versions • Improves accessibility for users and supports integration with external tools like OpenSim	• The system shall be compatible with Python version 3.8 to ensure proper OpenSim integration • The system shall run on both Linux and Windows operating systems	Shall
NFR-4	Should integrate with common Python simulation libraries like FMPy and OMPython	• Allows the framework to reuse validated components, support FMI-compliant models, and interact with Modelica environments • Prevents reinventing existing infrastructure.	• The system should provide a wrapper or interface for executing Functional Mock-up Units (FMUs) using FMPy. • The system should support the initialization and simulation of equation-based models via the OMPython API.	Should
NFR-5	Most follow good development practices	• Maintaining high-quality software engineering practices (modular design, documentation, unit tests) • Improves maintainability, extensibility, and reliability	• The system shall organize source code into clearly separated, reusable, and cohesive modules. • The system shall include automated unit tests for core components (e.g., component model handling, solvers, IO). • The system shall include user-facing documentation usage, examples, and API reference.	Shall
NFR-6	Code and models must comply with open-source licenses	• Open-source compliance ensures legal clarity for code reuse, collaboration, and distribution	• The source code shall be released under a well-known open-source license (e.g., MIT, BSD, GPL) with a corresponding license file (repo) • All third-party dependencies shall be documented and verified to comply with compatible open-source licenses.	Shall
NFR-7	Cite reused libraries properly	• Proper citation of reused libraries ensures compliance with licensing terms • gives credit to original authors • maintains transparency for users and contributors.	• The framework shall include a NOTICE or THIRD_PARTY_LICENSES file that lists all reused libraries along with their license type and version. • Documentation shall reference the original sources (e.g., repository URL, citation if required) of reused libraries and tools.	Shall

my_module_A.py

Data:

- Module Inputs: list(PhysicalValue)
- Module Outputs: list(PhysicalValue)
- Parameters: list(PhysicalValue)
- Initial Values: list(PhysicalValue)
- Internal Initial Values: list(PhysicalValue)

Functions:

Differential: rate equations $dy/dt = f(y,t)$
Algebraic: constraint equations $0 = g(y,z)$
Common: general computations
Internal: helper functions for internal use

my_module_B.py

...

my_module_C.py

...

my_module_D.py

...

Create Module Objects:

```
moduleA = Module("ModuleA", "my_moduleA.py")
moduleB = Module("ModuleB", "my_moduleB.py")
moduleC = Module("ModuleC", "my_moduleC.py")
moduleD = Module("ModuleD", "my_moduleD.py")
mod_list = [moduleA, moduleB, moduleC, moduleD]
```

Create Simulation Object:

```
sim = Simulation(mod_list)
```

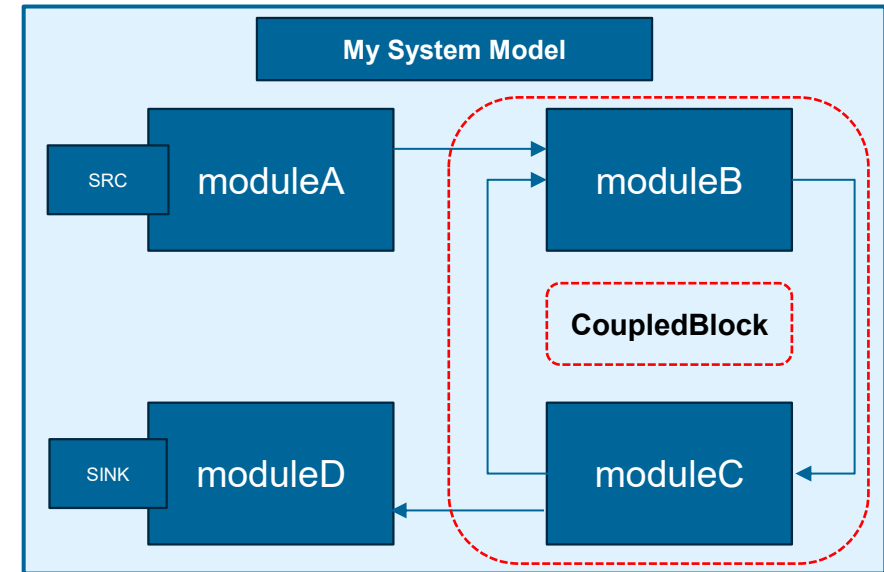
Define Module Connections

```
sim.connect("ModuleA", "A_out", "ModuleB", "B1_in")
sim.connect("ModuleB", "B_out", "ModuleC", "C_in")
sim.connect("ModuleA", "C1_out", "ModuleB", "B2_in")
sim.connect("ModuleA", "C2_out", "ModuleD", "D_in")
```

Run the simulation:

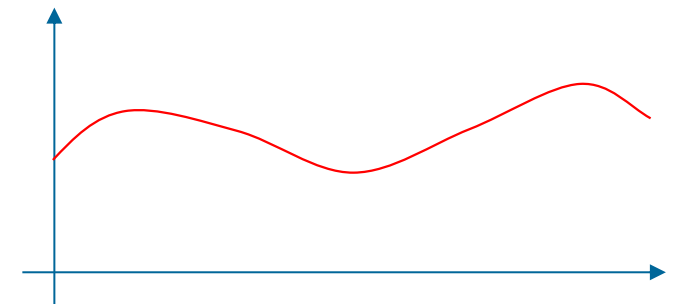
```
sim.run():
```

1. **_build_execution_order()**
 1. Detect strongly connected components (SCCs)
 2. Extract internal order of the SCCs
 3. Created CoupledBlock objects for SCCs
 4. Build Reduced Dependency Graph
 5. Sort to build final execution order
 6. Assign execution levels to modules
 7. Validate coupled blocks
2. **Initialize Modules in Topological Order**
 1. Propagate initial values from sources to destination modules
 2. Initialize each module / coupled block
 1. Initialize state variables
 2. Solve algebraic equations iteratively
 3. Validation and Finalization
 4. Initialize Derivatives and Output Buffers
3. **Perform Steps while $t < t_{end}$**
 1. Check Time Alignment for module execution
 2. Set Inputs
 3. Execute Modules / CoupledBlock objects
 4. Record History & Global Time Advancement

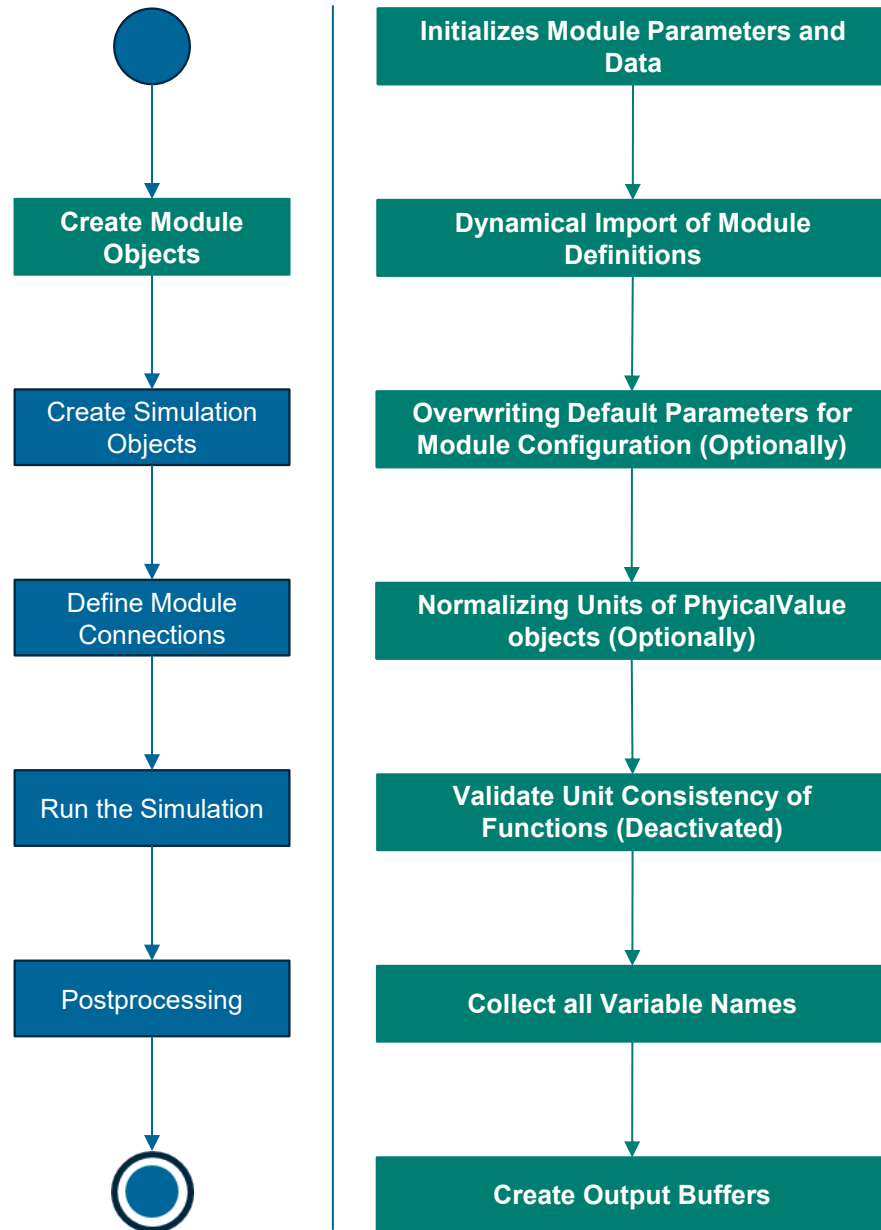


Extract Results & Postprocessing:

```
sim.export_history_to_excel()
results = sim.get_output_timeseries("moduleD", "D_out")
```



	Unit	PhysicalValue	Module	CoupledBlock	Simulation
Attributes	• Original string: representation of the unit in given form • Unit dictionary Example: {'kg': 1, 'm': 2, 's': -2}	• name: string • value: real, array • unit: Unit • original value • original unit	• Core Data: name, path, dt, solver methods • Module Definition: inputs/outputs, parameters, initial values, internal values • Runtime state variables: state variables, derivatives, algebraic variables, y_output, z_output, common output • Function Collection: differential, algebraic, common, internal	• Core Data: name, list of modules, module map for name to object, global module connections, time step, solver methods, level map • External Interface Mapping: input mapping, output mapping, inputs, outputs • Variable Management and Aliasing: variable to module dict, variable aliases dict, parameter dict • Runtime State Variables: state variables, derivatives, algebraic variables, y_output, z_output, common output • Function Collection: differential, algebraic, common, internal	• Core Data: list of modules, module map from module name to module object, dt, current time and end time, constant time step flag • Execution Management: execution order (list of modules and coupled blocks), flattened order list, module levels dictionary, sublevel dictionary for coupled block modules • Connection & Coupling Management: Connection dict, list of all detected and instantiated coupled blocks, module to coupled block dict • Data Storage & History: history dictionary for module output time series tracking • Monitoring and visualization: configuration dict for plot, matplotlib figure, list of axes, list of lines, list of monitor groups, monitor file
Methods	• Parse unit string into dictionary • overload operators: add, sub, mul, truediv, ... • Create a Unit object form a dictionary	• convert list to np.array • set /get value • set / get unit • string representation	• Module Loading and Setup: constructor, load description file, resolve inputs/outputs, gather the decorated functions, wrap physical values • Validate & Utilities: convert physical values to SI, validate unit consistency, string representation • Simulation execution: initialize, perform step, get outputs • Internal Function evaluation: • Gather state and algebraic values • Evaluate internal & common • Evaluate differential and algebraic	• Initialization and Setup constructor, validate modules' time step compatibility, register external connection, prepare the variables for common namespace, gather all module functions • Variable Resolution & Utilities resolve variable alias, gather state and algebraic variables, warn on wrong dependency, string representation • Validation & Safety Checks check the common function usage • Simulation Execution initialization, perform step, get outputs for external use and get outputs of all modules • Internal Function Evaluation • Evaluate internal & common • Evaluate differential and algebraic	• Initialization & Setup constructor, establish connection, set the time steps • Execution Order & Dependency Analysis build the execution order, assign levels to modules (standalone and within coupled blocks) • Simulation Execution initialize the simulation, set inputs of the involved modules, run the simulation, perform a time step, write outputs to history dictionary • Data Management & Export get timeseries output of module outputs, get the unit of a specific output, export the history to excel file • Visualization & Monitoring enable live monitoring, monitor results, plot outputs, visualize the system architecture with involved modules



```
class Module:
    def __init__(self, name, path, parameters=None, initial_values=None, dt=None, normalise_units=False):
```

Global Decorators:

- Marks functions as **differential, algebraic, common, and internal** equations and specifies their outputs
- `_function_type` allows to **discover all equations of the same type**
- Output specification allows the framework to **know which variables each equation computes** and **enables dependency analysis and proper execution ordering**

Class: *Module*

- Module class represents a modular, equation-based simulation components
- Defines **physical variables** for inputs/outputs, internal values, initial values, & parameters
- Creates a **dictionary to track current values y, z and their derivative dydt**
- Module definitions are **dynamically loaded from python scripts**
- Handles **time integration, algebraic loop resolution, and simulation stepping**
- Designed to simulate a single component/module as a self-contained unit

Data Organization:

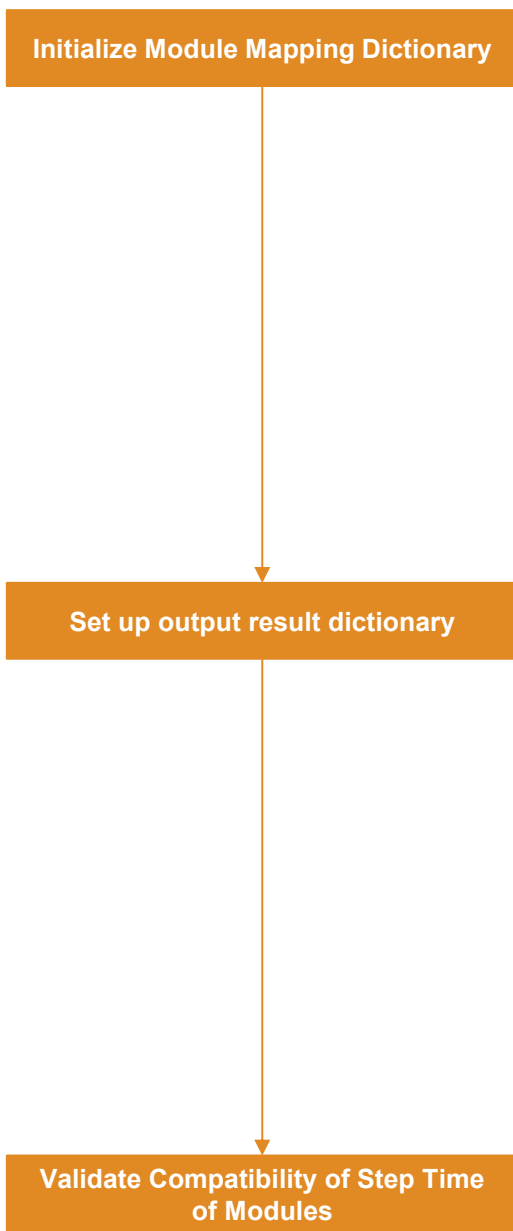
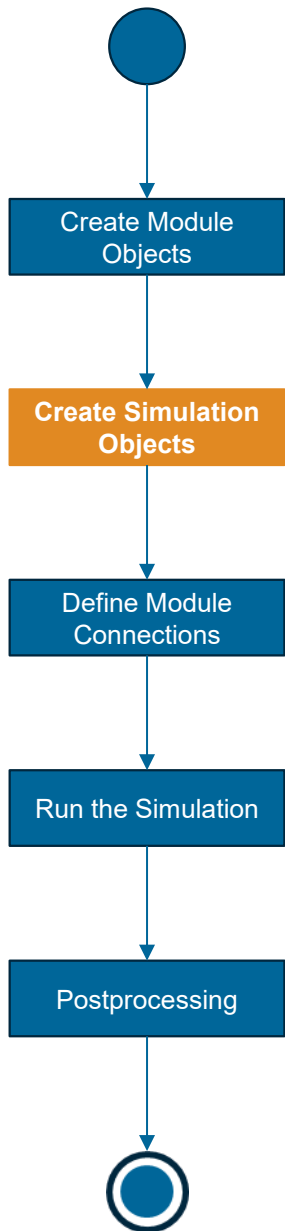
- **Parameters:** Fixed values that do not change during simulation
- **Initial values:** Starting conditions for state variables
- **Internal values:** Runtime variable that get updated each simulation step
- **Inputs/Outputs:** Interface variables for connecting this module to others

Key Features:

- **Dynamic Loading:** modules are loaded from files at runtime, enabling modular system design
- **Unit handling:** Optional SI unit normalization ensures consistent calculations
- **Solver Configuration:** Configurable ODE and nonlinear equations solvers (LSODA and hybr by default)
- **Parameter Overriding:** Constructor parameters can override default values from the module file

Strengths:

- Users can define a physical system in a python file with decorated functions and default parameters
- Flexible and maintainable: clear separation between definition in files and runtime state (in objects)



```
class Simulation:
```

```
def __init__(self, modules, dt=0.01, t_end=1):
```

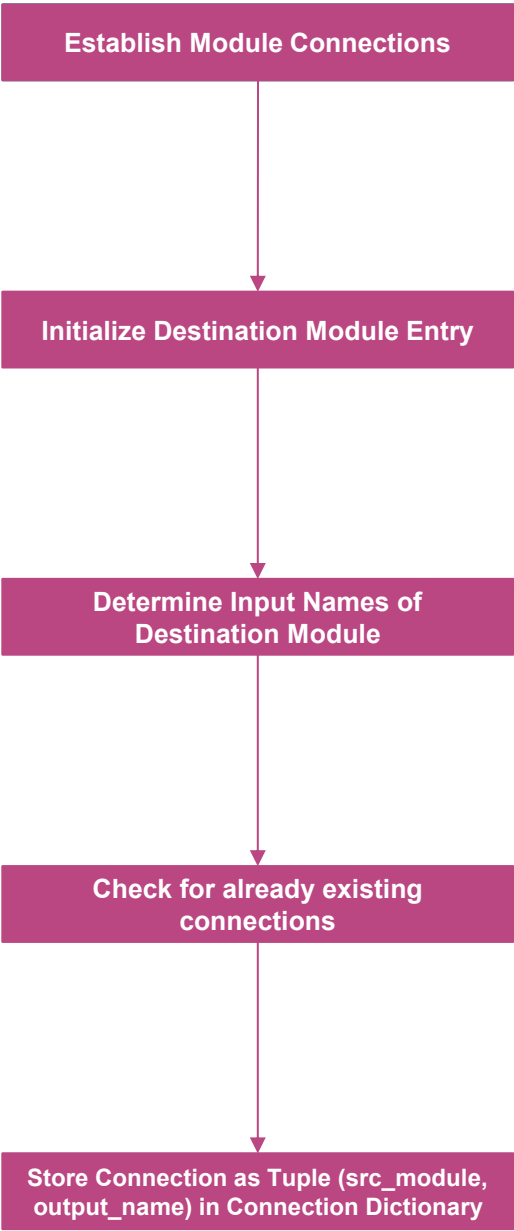
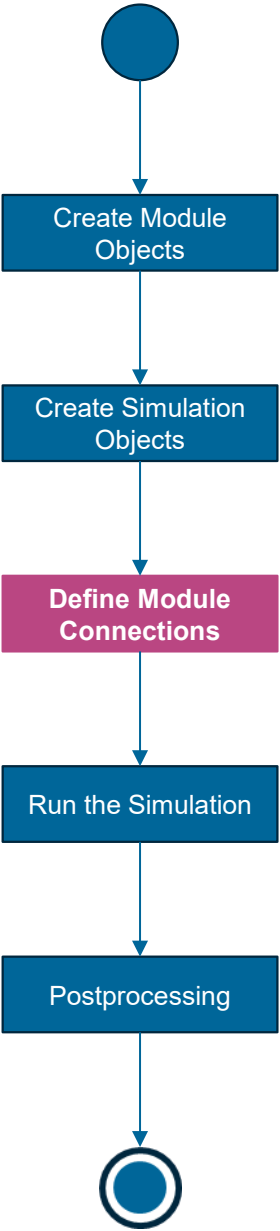
- Class is the main orchestrator for **multi-module dynamic system simulations**
- Manages **module connections, execution ordering, time stepping, and data collection**
- **Core framework architecture** and **execution logic** for the system simulation

Initialization of the Simulation Instance:

- **Module Management:** Creates a list of all modules and a dictionary that maps each module name to its instance
- **Connection system:** Dictionary that will store for each input the source module and output variable name
 - `dst_module → {input_name: (src_module, output_name)}`
- **Coupled Blocks:**
 - Maintains a list that stores all *CoupledBlock* objects that emerge through module connections
 - Maintains a dictionary that maps the modules to the *CoupledBlock* objects they belong to
- **Execution control:**
 - `self.execution_order`: Ordered list that will contain the standalone modules and coupled blocks in execution order
 - `self.modules_ordered`: Ordered list of all modules including their couplings
 - `self.module_levels`: Assigns integer levels to modules/coupled blocks for main execution order
 - `self.module_sublevels`: Assigns floating-point sublevels to modules within coupled blocks
- **Time management:** sets simulation parameters and validates time step consistency (time step, current time, end time)
- **Data collection:** initializes history tracking for all modules

Validating Compatibility of Involved Modules' Time Steps:

- Method handles the time step (dt) synchronization and validation across all modules in the simulation
 - Ensures **that all modules can work together with compatible time steps**
1. **Set default time steps** dt of simulation for modules that do not have any time step defined
 2. **Determines the smallest time step** among all modules at sets this as the global simulation time step dt
 3. **Checks for equal time steps** and sets flag which is used later for visualization
 4. **Validates the time step compatibility:** only multiples of the global simulation time step dt are allowed



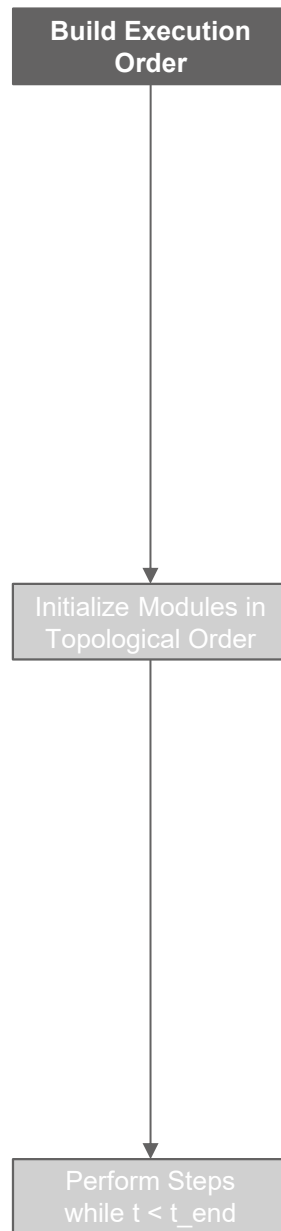
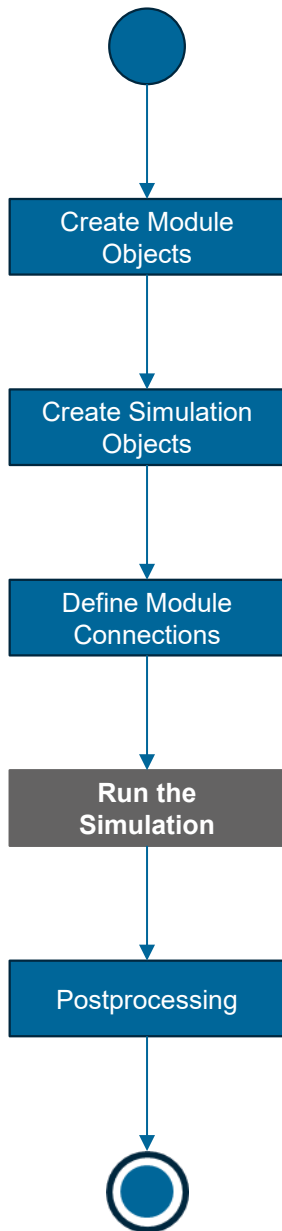
```
class Simulation:
def connect(self, src, output, dst, input=None):
```

- Method **establishes the connection between the modules** in the simulation by **linking outputs from source modules to inputs of destination modules**
 - src**: source module name (string) | **output**: name of the output signal from the source module
 - dest**: destination module name that receives the input | **input**: name of the input on the destination module

Step-by-Step Logic:

- Initialize Destination Module Entry**: creates an empty dictionary for the destination module if it doesn't exist yet; **each destination module gets its own dictionary** to store all its input connections
- Determine input name**: *input_name = input or output*
 - if no specific name is provided, it uses the output name as the input name
 - Example: `sim.connect("sensor", "temperature", "controller")` = connects `sensor.temperature` → `controller.temperature`
- Check for existing connections**: prevents multiple connections to the same input
 - Each input can only receive from one output**: if a connection already exists, it raises a descriptive error showing:
 - Which input is already connected, what is currently connected to, and what you are trying to connect to it
- Save the connection**: *self.connections[dst][input_name] = (src, output)*
 - Records the connection as a tuple: (source_module_name, output_name)
 - Creates the mapping used later during simulation execution

Example Usage	Resulting Data Structure
<pre># Connect sensor temperature output to controller temperature input sim.connect("sensor", "temp_out", "controller", "temperature") # Connect with same name for input/output sim.connect("controller", "control_signal", "actuator") # Equivalent to: sim.connect("controller", "control_signal", "actuator", "control_signal")</pre>	<pre>{ "controller": { "temperature": ("sensor", "temp_out") }, "actuator": { "control_signal": ("controller", "control_signal") } }</pre>



```
class Simulation:
def _build_execution_order(self):
```

Purpose:

- Algorithm determines the proper execution sequence for modules
- Handles simple dependencies and feedback loops
- Core orchestrator that creates a valid execution order for all modules while properly handling circular dependencies (feedback loops) by grouping them into *CoupledBlocks* objects

Step 1: Build Full Dependency Graph

- Creates a **directed graph** representing all modules dependencies
- **Nodes:** Module Names | **Edges:** Data Flow Connections
- If sensor outputs to controller: **edge from sensor → controller**

Step 2: Detect Strongly Connected Components

- Identifies groups of **modules with circular dependencies**
- Single Module SCC: No feedback loops
- Multi-Module SCC: Contains feedback loops that need special handling
- Example:
 - $A \rightarrow B \rightarrow C$ (3 separate SCCs: [A], [B], [C])
 - $D \rightarrow E \rightarrow D$ (1 SCC with feedback: [D, E])

Step 3: Extract Internal Order for each SCC

- Key Algorithm: Reversed DFS Post-Order Traversal
- DFS Post Order: Visits the children before parents
- Reversed: Gives us a topological-like ordering even with cycles
- Result: A "reasonable" execution order that minimizes feedback dependency violations

Step 4: Create Coupled Blocks for Multi-Module SCCs

- Encapsulate feedback loops in specialized blocks
- CoupledBlock: Container that handles the iterative solution of algebraic loops
- Mapping: Track which modules belong to which coupled block
- Name: deterministic naming consistency

Step 5: Build Reduced Dependency Graph

- Create high-level dependency graph between modules and coupled blocks
- Nodes: Individual modules + Coupled Block names
- Edges: Only inter-group dependencies (not internal to coupled blocks)
- Result: Acyclic graph at the top level

Step 6: Topological Sort for final order

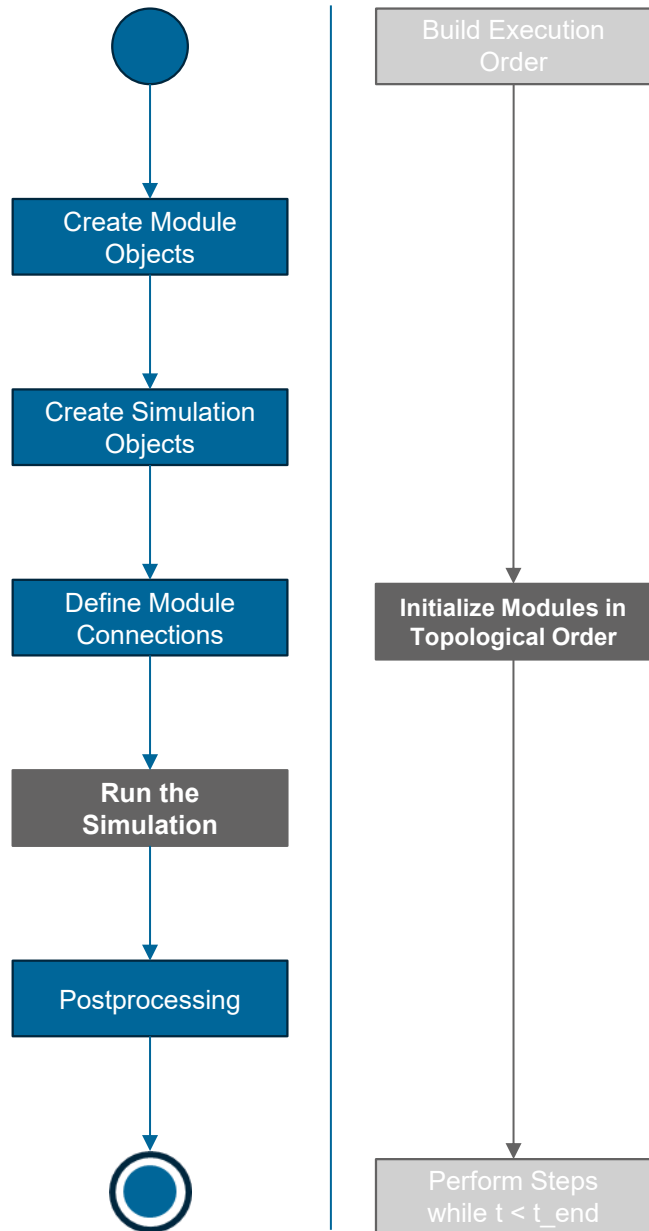
- Determine the final execution sequence
- Topological Sort: Ensure dependencies are satisfied
- Mixed Objects: List contains both Module and *CoupledBlock* instances
- Guarantee: No Module executes before its dependencies

Step 7: Assign Execution Levels

- Execution levels: modules/coupled blocks in topological order
- Sub-levels to modules within CoupledBlock for internal ordering

Step 8: Validate Coupled Blocks

- Ensure that CoupledBlocks don't have valid configurations that could cause algebraic loops



```
class Simulation:
    def _initialize(self):
```

Purpose:

- Method is responsible for setting up the initial state of all modules in the simulation before the actual time-stepping begins
- Method processes modules in their predetermined execution order, which is crucial for proper initialization
- Execution order contains coupled blocks and standalone modules
- Ensures that when a module / block is initialized, all its input dependencies have already been processed

Step 1: Value propagation

- Propagates initial output values from source modules to destination module inputs
- For each modules, look up its input connections and retrieves the current output values from connected source modules
- Sets the input values on the current module

Step 2: Module / CoupledBlock initialization

- Calls each module's own initialization method to set up its internal state calling `mod.initialize()`

Within `mod.initialize()`:

Step 1: Initialize State Variables

- Method first populates all state variables which are the outputs of the differential equations with their user defined initial values
- Validates that every state variable has a corresponding initial value, preventing the simulation from starting with undefined states

Step 2: Solve Algebraic Equations Iteratively

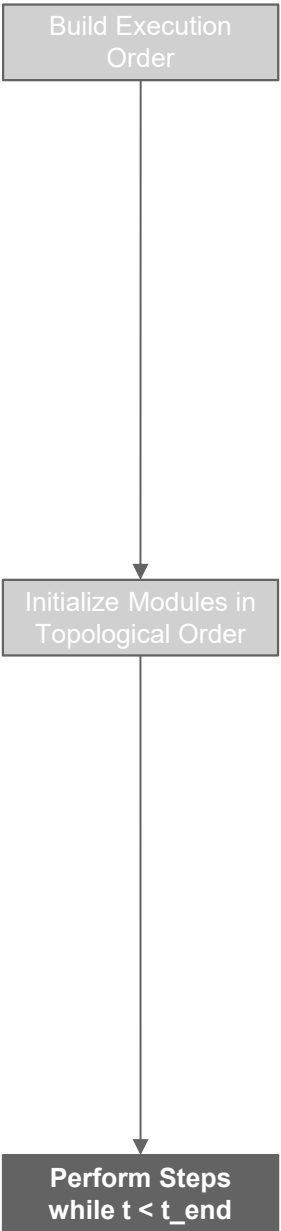
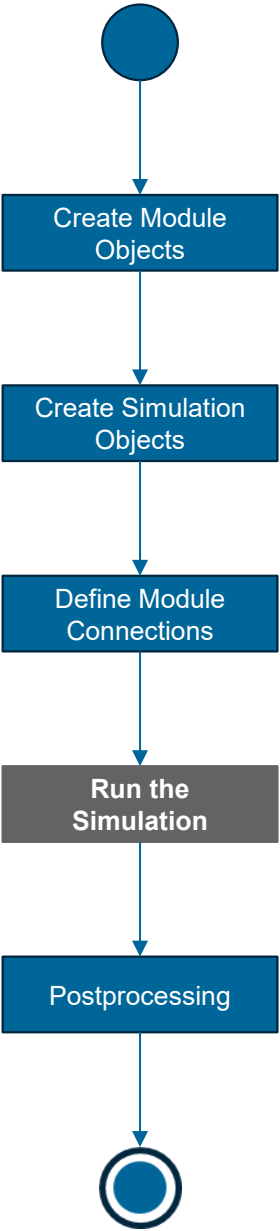
- Implements a dependency resolution algorithm
- Iterative solving: loops through algebraic functions multiple times
- Dependency checking only evaluates functions when all their inputs are available
- Deadlock detection: breaks if no progress is made

Step 3: Validation and Finalization

- Method ensures all algebraic variables were successfully computed
- If any remain unsolved, it indicates missing dependencies or circular references

Step 4: Initialize Derivatives and Output Buffers

- Sets derivative estimates to zero (will be computed in first simulation step)
- Prepares output buffers for external access



```
class Simulation:
def step(self):
```

Purpose:

- Method is the core time-stepping function that advances the simulation by one time increment
- Handles multi-rate simulation where different modules can have different but compatible time steps

Step 1: Check Time Alignment

- Only proceeds if module's time step aligns with the current simulation time

Step 2: Set Inputs

- Propagates current output values from connected source modules
- Ensures the module has the latest input data before execution
- Handles both regular modules and CoupledBlocks

Step 3: Execute Module / Coupled Block

- Calls the module's internal stepping function
- Advances the module's internal state by one time step
- Computes new output values based on current input state

Step 4: Record History

- Stores the module's current output values with timestamp
- Enables post simulation analysis and plotting

Step 5: Global Time Advancement

- Advances the global simulation time by the smallest time step dt
- Rounds to 10 decimal places to avoid floating point accumulation errors
- Ensures the simulation progresses in discrete, predictable increments

Execution Order Importance:

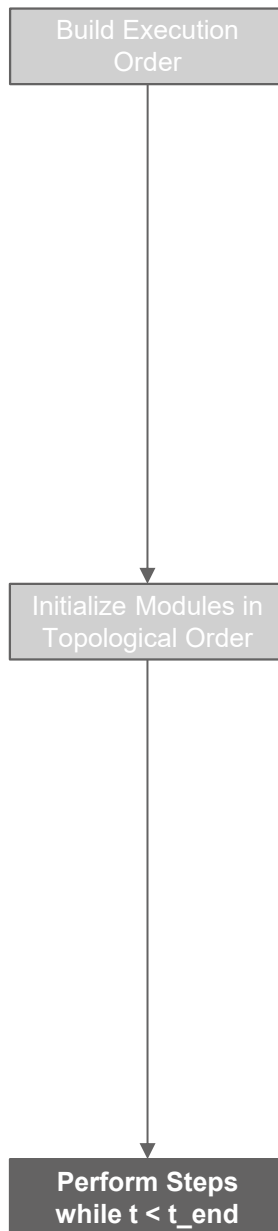
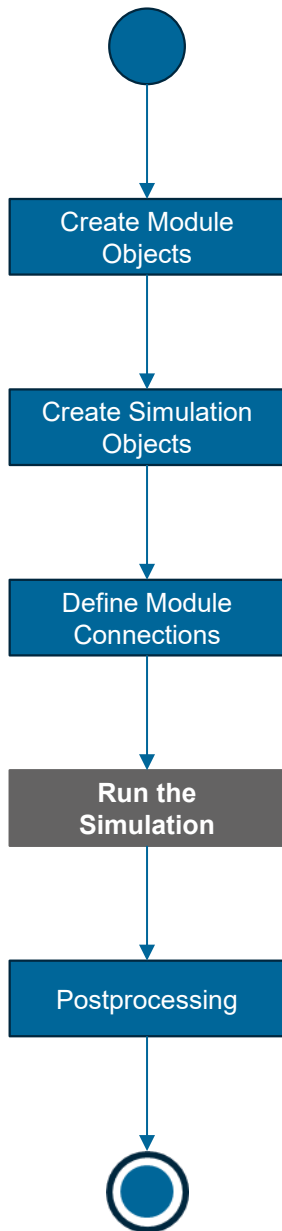
- Method processes modules in self.execution_order
- Data dependency: modules receive inputs from previously executed modules
- Causality: Ensures proper cause-and-effect relationships
- Feedback handling: Coupled Blocks handle circular dependencies correctly

Key Design Features:

- Floating-Point Precision Handling:** large scale factor (10^{10}) and rounding operations reduce floating-point precision issues
- Multi-Rate Support:** Modules only execute when needed, allows systems with varying time scales
- Deterministic Execution:** fixed execution order through time alignment
- Modular Design:** Each module / coupled block is responsible for its own internal stepping logic within mod.step()

Multi-Rate Simulation Example:

Module Time Steps	Execution Timeline
- ModuleA: dt = 5 ms - ModuleB: dt = 10 ms - ModuleC: dt = 20 ms Global: dt = 5 ms	- t = 0 ms: A, B, C execute - t = 5 ms: A executes - t = 10 ms: A, B execute - t = 15 ms: A executes - t = 20 ms: A, B, C execute



```
class Module: / class CoupledBlock:
def step(self):
```

Purpose:

- Method is the core simulation engine **that advances the module's state from time t to $t + dt$**
- Implements a numerical integration scheme that handles both differential equations **ODEs and algebraic constraints (DAE)** in a coordinated manner

Step 1: Evaluate Internal Functions

- Internal functions are helper functions for internal calculations
- Updates private module calculations based on current state before proceeding with main calculations

Step 2: Evaluate common function

- Compute shared outputs and intermediate values using updated internal state
- Example: calculate power output, status flags., or diagnostic values that other modules need

Step 3: Store current outputs

- Preserve current state as outputs for external access while the method computes the next timestep internally
- Key insight: **output values represent the current time** while internal calculations proceed to predict next timestep

Step 4: Integrate Differential Equations

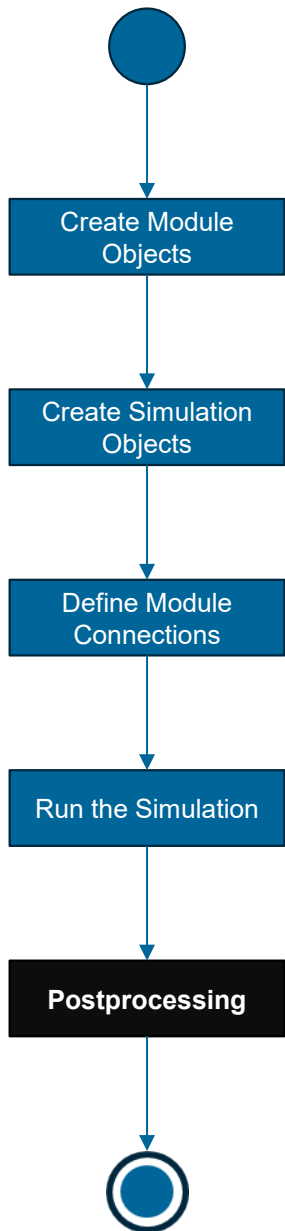
- $dydt$ closure: evaluate derivatives at intermediate time points during integration
- State coupling: **uses current algebraic variables $self.z$ during integration**
- Result extraction: takes final predicted state $y[k+1]$**

Step 5: Solve Algebraic Constraints

- Scenario A: DAE system**
 - solves for both $dydt[k+1]$ and $z[k+1]$ simultaneously
 - ensures derivatives are consistent with algebraic constraints at new state
 - residual checks: $f(y[k+1], z[k+1]) - dydt[k+1] = 0$ and $g(y[k+1], z[k+1]) = 0$
- Scenario B: pure algebraic system**
 - only solves for $z[k+1]$ given predicted $y[k+1]$
 - simpler case where no derivative consistency is needed

Mathematical Interpretation:

- Method implements a predictor-corrector schema for differential algebraic equations
- Predictor Phase (step 4):**
 - $y[k+1] = \text{integrate}(dy/dt = f(t, y, z[k]), \text{ from } t \text{ to } t+dt)$
- Corrector Step (step 5):**
 - Solve: $\{ f(y[k+1], z[k+1]) - dydt[k+1] = 0 \}$
 $\{ g(y[k+1], z[k+1]) = 0 \}$
- Ensures that the **derivatives and constraints are consistent** at new timestep



```
class Module:  
    def get_outputs(self):
```

Purpose:

- Method provides the external interface for **accessing module results** by
 - Collecting computed values **from different internal sources**
 - Packaging them as **properly formatted objects with correct units**
- Method serves as bridge between **internal computation and external access**

Value Source Hierarchy

- Method searches for output values in three internal dictionaries with a specific precedence
 - State variables [y_output]**
 - values from differential equations continuous dynamics
 - Examples: capacitor voltage, motor_speed, temperature
 - Algebraic Variables [z_output]**
 - values from constraint equations - instantaneous calculations
 - Example: current, power, efficiency
 - Common function results**
 - values from @common functions - computed outputs
 - Example: status flag, diagnostic values, derived quantities

Integration with Module Declaration

- Method relies on the module's **[MODULE_OUTPUTS] declaration**:
- Method ensures
 - only declared outputs are returned
 - all outputs have correct units
 - missing outputs trigger clear error messages

```
class CoupledBlock:  
    def get_all_outputs(self):
```

Purpose:

- Method collects and organizes all outputs from internal modules within a coupled system
- Method creates a structured dictionary for history logging in simulations
- Method iterates through all modules and their outputs
- Creates a nested dictionary where the outer keys are module names and the inner keys are the output names
- Each output is represented as a PhysicalValue object containing the name, actual value, and unit information

Data Structure

- Method uses **defaultdict(dict)** to **automatically create nested dictionaries**
- When you access grouped[mod.name], if mod.name does **not exist yet as a key, defaultdict automatically creates an empty dictionary for it**
 - Eliminates the need for manual key existence check
- For each output: method constructs a fully qualified name mod_name.name and searches for the actual value across three different storage locations
 - y_output, z_output, common_output
- Final return converts the defaultdict back to a regular dictionary** that prevents external code from accidentally creating new entries

Usage: within _write_history() and then _get_output_timeseries()

- Handles history logging for simulation modules (standalone modules and coupled blocks)
- Checks the type of the object to determine the method for extracting outputs
- CoupledBlock: calls _get_all_outputs
- Module: calls _get_outputs() wraps the result in a dictionary

ID	User Requirement	Fulfilled?	Justification	Improvements
UR-1	The framework shall support the component-based modeling of multi-domain systems .	Fulfilled	Module class provides a base for component modeling, - Each module encapsulates inputs, outputs, state variables, and behavior via functions	
UR-2	The framework shall support the use of units for all physical quantities .	Fulfilled	- Includes a class PhysicalValue that encapsulates a value, unit, and a name Units are handled using the Pint library - Each Module handles PhysicalValue objects	- Unit consistency checks for modules to validate dimensional correctness of functions - Shift more of the unit handling to the Pint module
UR-3	The framework shall allow the creation of system models by connecting subsystems .	Partially	- Subsystems must be defined as Modules, modules cannot be combined in virtual or atomic subsystems - Simulation only handles modules or CoupledBlocks for simulation steps - The system model hierarchy is flat, hierarchical systems with modules within different levels are not supported	- new class Subsystems that encapsulates multiple modules - Provide templates for creating subsystems from modules - Provide explicit interfaces
UR-4	The framework should allow the acausal equation-based modeling of the component behavior.	Not Fulfilled	Inputs and Outputs are explicitly defined within the module definition No symbolic equation representation No computational causality analysis	- Introduce symbolic support via SymPy - Perform structural analysis (BLT decomposition, Pantelides algorithm) - OpenModelica Interface for equation-based sub models
UR-5	The framework shall enable the causal equation-based modeling of the component behavior.	Fulfilled	Causality is given by defining inputs and outputs for each function within the module definition Differential, common, and algebraic functions evaluate outputs from known inputs - Simulation engine ensures that the input values are resolved before function calls	
UR-6	The framework shall support the definition of component behavior using discrete-time or hybrid dynamics .	Not Fulfilled	No explicit event-handling, switching logic, or reinitialization logic - No built-in support for zero-crossing detection or mode transitions - Execution explicitly defined by time step, no intermediate execution or rollback due to events	- Event-Handling with zero-crossing detection and reinitialization logic for continuous states - State-based triggers
UR-7	The framework should provide templates for the creation of equation-based component models.	Partially	- Framework provides templates for the generation of modules (scripts with decorated functions and PhysicalValues parameters) - No reusable templates since every component must be defined from scratch	- Library of commonly used elements - Template inheritance or parameterization
UR-8	The framework shall allow the reuse of components and subsystems with different configurations.	Partially	- The subsystems allows the reuse of modules(components) with different configurations through overwriting default parameters - Subsystems (encapsulate several modules) are not yet part of the framework	- Allow named subsystem definitions (user-defined coupled blocks) - Allow instantiation of coupled blocks

ID	User Requirement	Fulfilled?	Justification	Improvements
UR-9	The framework shall enable the integration of subsystem models developed using different modeling tools.	Not Fulfilled	• No support for FMU import via FMPy • No API Layer for OpenSim/FEM • All modules must follow the custom internal structure (decorators / inputs and outputs as physical values)"	• Integrate FMPy for Loading FMUs (ME and CS) • Provide wrapper interface for FMUs, OpenSim and FEM tools
UR-10	The framework should support the combination of equation-based and other heterogeneous sub models.	Not Fulfilled	• Heterogeneous modeling is not yet available • No infrastructure to load or connects models from external tools • All modules are expected to follow the internal solve pipeline • not tailored for hybrid integration	• Generic wrappers for external modules • Define simulation role interfaces that handle step(), initialize(), get_outputs() uniformly for different backends • Allow modules with different causalities and time semantics to be composed together • Allow tool-specific configurations in the module or system level

ID	User Requirement	Fulfilled?	Justification	Improvements
UR-11	The framework shall enable the numerical simulation of hybrid systems with both continuous and discrete-time behavior.	Partially	• The framework makes a distinction between differential and algebraic functions using decorators, it uses a discrete time step management inside the simulation engine • Executes modules only when their local dt aligns with the global simulation time • There is no event handling, reinitialization, or solver rollback • No zero-crossing detection or discrete/continuous state boundary detection • Step alignment is not suitable for complex hybrid interaction	• Support discrete events triggering continuous resets • Introduce event detection • Support reinitialization of state variables on event trigger • Abstract state management to distinguish discrete vs. continuous states
UR-12	The framework shall allow the user to configure and select solver types and step sizes for simulation.	Partially	• Framework enables global simulation time step dt when instantiating the simulation • Solver options (tolerances) are not exposed • No abstract solver interface	• Expose solver arguments as keyword arguments to customize integration tolerances • Encapsulate Solver Adapter inside a class for flexibility • Distinguish continuous and discrete solver policies

ID	User Requirement	Fulfilled?	Justification	Improvements
UR-13	The framework should provide a visual representation of the system model and its hierarchical structure .	Fulfilled	- Framework supports the visualization via visulaise() method of simulation class Uses Graphviz to draw the graph	- Differentiate module types with node shapes or colors (e.g., source/sink, algebraic) - Cluster CoupledBlocks visually to reflect encapsulation - Add hierarchical expansion/collapse
UR-14	The framework should allow the user to observe simulation progress through graphical and textual output.	Fulfilled	- Framework provides graphical monitoring and live plotting - Live plotting can be configured via dictionary - Allows the export of monitoring plot as PNG Textual logs are provided	
UR-15	The framework should allow the user to compare simulation results across different configurations .	Partially	- Capabilities are available but comparison logic is not abstracted into a higher-level API - Simulation results are stored in self.history and can be exported to excel for further analysis No built-in mechanism to run & compare multiple configurations automatically No built-in support for plotting results of multiple runs	- A lightweight API wrapper could enable structured comparison workflows - e.g., loop over configs → store labeled results → plot overlay
UR-16	The framework should support result export and post-processing functionality.	Fulfilled	- The framework allows the result export to excel format for post processing. Only excel format and default as python output are supported. - No statistical pre-analysis is performed on the results	

Modeling:

Capabilities / Strengths	Limitations / Drawbacks
• Component-based Modeling: Modules are used to represent system components • Causal modeling is naturally supported via data flow and connections • Unit Handling: uses Pint to define, normalize, and operate on physical units • Reusable Modules: Components can be reused with different parameterizations using constructor arguments. • Multi-rate simulations: Modules can specify individual time steps; hybrid behavior is supported through mixed execution.	• No true subsystems available and thus limited encapsulation and modularity, nested modules or subsystem • Modules can only be defined configured purely in Python, no domain specific definition or specification language • No support for hybrid DAE systems with continuous and discrete states

Simulation / Solver:

Capabilities / Strengths	Limitations / Drawbacks
• Custom solver using solve_ivp for continuous-time integration and root() for algebraic equations • Supports multi-rate systems: Modules can define custom time steps (support for mixed discrete and continuous updates) • Algebraic loops handled through CoupledBlock abstraction and numerical root-finding • Execution graph construction uses topological sort and SCC detection to respect dependencies.	• No solver configuration interface exposed to the user • No option to select integration methods or tolerances • Only single-threaded, single-core simulation • no support for parallelization or co-simulation.

Heterogeneity:

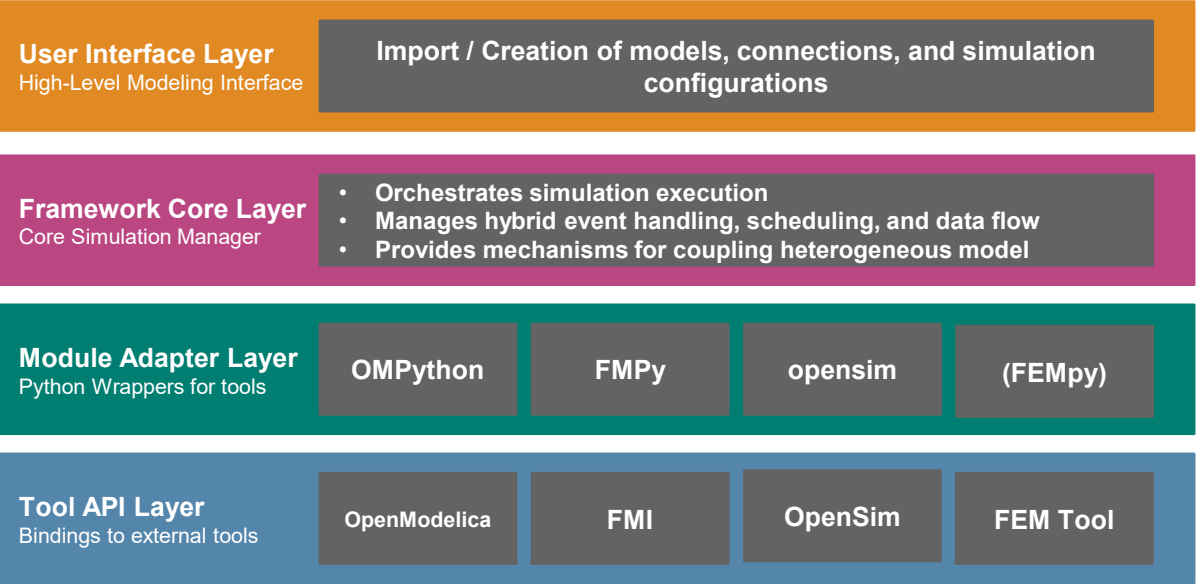
Capabilities / Strengths	Limitations / Drawbacks
• Simulation-based modularity • Would allow for the integration of external models.	• No support for external model integration yet (e.g., no FMI, OMPython, or tool-coupling interfaces) • Only causal equation-based modules currently supported • Heterogeneous models (e.g., FMUs, FEM solvers) cannot yet be integrated. • No interface abstraction layer for subsystems • No FMI/external integration

User Interaction:

Capabilities / Strengths	Limitations / Drawbacks
• Live output monitoring with matplotlib (configurable subplots, dynamic updates) • Visual representation of system structure via Graphviz-based visualise() method (ranked, annotated system graph) • Export and post-processing: Excel export of results with optional single-sheet view • Configurable plotting of results by groups and subplots.	• Visualization is limited to matplotlib and Graphviz • No simulation dashboard or inspection tools • Error messages and logging are minimal and not standardized.

- Core Idea:
- Wrapping established simulation tools like OpenModelica via OMPython, OpenSim via opensim package, FEM Python APIs, and using FMUs for model import to build a layered system simulation framework
 - The approach is to use tools (e.g., OpenModelica) that already solve the hard problems: **symbolic analysis, causalization, DAE solving, hybrid simulation, etc.**
 - **Frontend:** Parsing, replacements, modifications, instantiations, lookups
 - **Backend:** Causalization, index reduction, simplification, solvability
 - Approach leverage robust, validated open-source ecosystems (OpenModelica, PyFMI, FMPy, OpenSim)
 - Avoid to reinvent functionality that is available in open-source modules
 - Framework would focus on the integration of existing packages and orchestrating heterogeneous models for system simulation rather than implementing and optimizing algebraic loop detection or DAE solvers

- Critical / Open Aspects:
- Clear domain specific language (DSL) or user interface to define model definition
 - Textual as Python classes, configuration based YAML, JSON, XML, Python DSL?
 - Support for non-FMU tools is the “real” difficulty as they cannot rely on FMI Standard
 - Requires special wrapper adaption that are compatible with FMI
 - Data exchange and orchestration: master algorithm must define time stepping, event handling, data exchange across co-simulated subsystems
 - Toolchain setup: document and automate the toolchain to set all packages and dependencies correct up
 - Some Python packages might require specific Python versions
 - Error handling, logging and tracing might get complex if several tools are involved
 - Partial symbolic preprocessing: some symbolic insight on system structure might be beneficial before involving other tools (e.g., OpenModelica) within the process
 - Requires deep understanding of the involved tools and their Python packages (not purely Python)



Comparison vs. Existing Framework		
Aspect	Existing Framework	Alternative Approach
Solvers	Manual selection via scipy	Provided by OpenModelica/FMUs
Unit Handling	Via Pint, but no validation	Provided by OM but needs extensions
Modularity	Only flat system hierarchy	Full support via OM and FMUs
Hybrid System	Not supported	Full support through OM and FMI 3.0
Heterogeneous Integration	Not supported	Core of the framework approach
Usability	Only Python Module definitions	Several model sources, OM STL
Scalability	Limited due to central simulation	High through FMU encapsulation
Robustness	Suited for small and medium systems	Can use OMC's robust model checking and debugging support

Option 1: Adapting, Extending, Refactoring Existing Framework Approach

- 1. **Modular Refactoring**
 - Improve component separation (Module, Simulation, CoupledBlock, PhysicalValue)
- 2. **Architectural Adaptations**
 - Introduce hierarchical modeling: model representation via subsystems
 - Add abstraction layers: simulation component, boundaries between components
- 3. **Implementing Required Capabilities**
 - Add discrete event support
 - Introduce FMI Importer and wrap as modules
 - Add OpenSim Wrapper
- 4. **Ensure Unit Consistency and Validation**
 - Use pint module for full validation and unit propagation

Option 2: Following Alternative Wrapper & API-based Framework Approach

- 1. **Tool Interface Feasibility Study**
 - 1. Develop deep understanding for OMPython, FMPy, and OpenSim Python API
 - 2. Determine which toolchains can be controlled via Python (compile, simulate, extract results)
- 2. **Define Simulation Kernel**
 - 1. Which part provides the backend functionality and solvers for hybrid DAEs?
 - 2. Define abstraction for a simulation components (FMU, OpenModelica, OpenSim, FEM)
- 3. **Layered Architecture Design:**
 - 1. Clear separation of tool API, adapter modules for each tool, system / subsystem composition layer, simulation engine and coordinator, modeling interface for users
- 4. **Build First Vertical Slice**
 - 1. Simple Prototype: 1 x OpenModelica FMU, 1 x OpenSim model, run co-simulation
- 5. **Develop Master Algorithm Logic**
 - 1. handle step synchronization and events, manage data flow between models

Common First Steps

- 1. **Finalize the Requirement List**
 - Consolidate user/system/non-functional requirements into markdown document tagged with priorities (shall / should / nice to have)
 - Create traceability matrix for follow up and link to implementation and tests within repo
- 2. **Develop Evaluation Criteria and Metrics**
 - Define clear evaluation criteria for each requirement
 - How to measure functional, non-functional, and usability requirements?
- 3. **Setting up / Adapting Git Repo Structure:**
 - Adding directories: src/, tests/, docs/, examples/, etc.
- 4. **Finalize Demo Models** for Model-Driven Development and Testing
 - **Simple:** Existing Electric Circuit BPF/LPF
 - **Intermediate:** Pendulum with discrete events and tool integration
 - **Advanced:** Complex active orthoses coupled with musculoskeletal system)

Comparison: Refactor Existing vs. New-Wrapper-Based Approach

Aspect	Option 1	Option 2
Initial effort	Medium	High
Current Code Usage	High	Minimal
Integration potential	Medium (some larger adaptations for OpenSim/FMUs required)	High (tool integration core aspect)
Risk	Lower (known base)	High (tool interoperability challenges)
Innovation Potential	Medium (Simplified reimplementaion of hybrid DAE system solvers)	High
Timeline fit	Lower risk	Higher risk / more milestones

1. Sommerville, I. (2011). *Software Engineering* (9th ed.) [Book]. Addison-Wesley.

Module: *physical_value.py*

Purpose:

- Defines classes and functions that enable the physical modeling of modules using **PhysicalVariable objects consisting of a name, a value, and a unit**
- Defines a **Unit Class**, functions for unit handling and a **PhysicalVariable Class**
- Uses **the UnitRegistry of the Pint module** to manage unit definitions, conversions, and relationships
- Essential for **ensuring unit consistency via module interfaces** and allows dimensional analysis

Global Functions:

normalize_unit_string():

- Maps the Pint unit names for the seven SI unit names to their corresponding standard abbreviated forms (**e.g., kilogram = kg, second = s, etc.**)
- Normalizes mathematical operators in the unit expression for consistency
- Could be split in two functions for replacing and normalization of the operators

convert_to_si():

- converts a unit-aware quantity to its base SI representation with abbreviated SI unit and formatted unit string representation
- Example: (1, "kJ") -> (1000.0, "kg*m^2/s^2")

Class: *Unit*

- Method `__parse_unit__()` transforms the string representation into a dictionary representation with **SI units as keys and exponent as values** ("kg*m^2/s^2" -> {'kg':1, 'm':2, 's':-2})
- Implements operator overloading for performing unit arithmetic (`_add__`, `_mul__`, etc.)
- Re-implements major functionalities of the Pint module class and could thus be changed to a lightweight Unit function

Class: *PhysicalValue*

- Represents a physical quantity defined by a **name (string), a value, and a unit**
- Ensures that the **unit is always a Unit object**
- **Preserves the original value and unit** before normalization may be applied

Module: *module.py*

Global Decorators:

- Marks functions as **differential, algebraic, common, and internal** equations and specifies their outputs
 - `_function_type` allows to **discover all equations of the same type**
 - Output specification allows the framework to **know which variables each equation computes and enables dependency analysis and proper execution ordering**

Class: *Module*

Purpose:

- Module class represents a modular, equation-based simulation components
- Defines **physical variables** for inputs/outputs, internal values, initial values, & parameters
- Module definitions are **dynamically loaded from python scripts**
- Handles **time integration, algebraic loop resolution, and simulation stepping**
- Designed to simulate a single component/module as a self-contained unit

Data Organization:

- **Parameters:** Fixed values that do not change during simulation
- **Initial values:** Starting conditions for state variables
- **Internal values:** Runtime variable that get updated each simulation step
- **Inputs/Outputs:** Interface variables for connecting this module to others

Key Features:

- **Dynamic Loading:** modules are loaded from files at runtime, enabling modular system design
- **Unit handling:** Optional SI unit normalization ensures consistent calculations
- **Solver:** Configurable ODE and nonlinear equations solvers (LSODA and hybr by default)
- **Overrides:** Constructor parameters can override default values from the module file

Strengths:

- Users can define a physical system in a python file with decorated functions and data tables
- Flexible and maintainable: clear separation between definition in files and runtime state (in objects)

Module: *simulation.py*

Purpose:

- Class is the main orchestrator for **multi-module dynamic system simulations**
- Manages **module connections, execution ordering, time stepping, and data collection**
- **Core framework architecture** and **execution logic** for the system simulation

Initialization of the Simulation Instance:

- **Module Management:** Creates a lookup map and stores all individual modules
- **Connection system:** initializes empty connection dictionaries for inter-module communication
- **Execution control:** structures for determining execution order / grouping coupled modules
- **Time management:** sets simulation parameters and validates time step consistency
- **Data collection:** initializes history tracking for all modules

Execution Order:

1. **Builds a directed graph from all module connections**
 - Uses NetworkX to find strongly connected components (SCCs)
 - Single-nodes SCCs represent sequential modules
 - Multi-node SCCs represent coupled systems
2. **Creates *CoupledBlock* objects for each multi-module strongly connected component**
 - Instantiates *CoupledBlock* Object for each multi-module SCC
 - These blocks handle simultaneous equation solving for cyclically dependent modules
3. **Topological Ordering**
 - Creates a **reduced dependency graph between modules and *CoupledBlocks* objects**
4. **Topological Sort:**
 - Performs topological sort to **determine execution sequence**
 - Ensures the **system remains solvable (no cycles)**

Execution Level Assignment:

- Method: `_assign_levels()` creates a **hierarchical execution order**
- **Top-Level Assignment:** assigns integer levels to **modules and coupled blocks** bases on their dependencies
- **Sub-level Assignment:** **Within coupled blocks** assigns fractional sublevels (e.g., 3.1, 3.2, 3.91) to handle internal dependencies and feedback loops

Setting Up Initial State:

- Method: `_initialize()` is responsible for **setting up the initial state of all modules** in the simulation before the actual time-stepping begins
1. Method processes modules in **their predetermined execution order** (crucial for proper initialization)
 - Ensures that when a module is **initialized**, all its input **dependencies have already been processed**
 1. Propagates initial **output values from source modules to destination module** inputs
 2. Calls **each module's** own initialization method to **set up its internal state**

Time Stepping:

1. **Check Time Alignment:**
 1. Only proceeds if module's time step aligns with the current simulation time
2. **Set Inputs**
 1. Propagates current output values from connected source modules
 2. Ensures the module has the latest input data before execution
 3. handles both regular modules and *CoupledBlocks*
3. **Execute Module / Coupled Block**
 1. Calls the module's internal stepping function
 2. Advances the module's internal state by one time step
 3. Computes new output values based on current input state
4. **Record History:** Stores the module's current output values with timestamp
5. **Global Time Advancement:** Advances the global simulation time by the smallest time step

Module: *coupled_block.py*

Purpose:

- Creates a composite simulation **component that contains multiple individual modules**
- Handles **complex interconnections** between them while presenting a **unified interface** to simulation system
- Designed to manage both the **internal coupling between modules and the external interfaces** that connect to other parts of the simulation
- **Validates compatibility of all constituent modules**: they must share the same time step and ideally use the same numerical methods
 - `_validate_dt()`: enforces strict time step consistency using high precision rounding to avoid floating point comparison issues
 - `_validate_methods()`: warns about mismatched solver preferences but allows execution to continue

Set Up Phases:

1. **Establishes external interface** through `_register_external_connections()`
 - Analyzes the global connection map to determine which module inputs need external data and which module outputs should be exposed
 - Creates the boundary of the coupled block
2. Method `prepare_variables()` using the dot notation ("motor.velocity") **flattens all module variables into a unified namespace**
 - Maintains traceability back to the source modules
 - Method handles all variable types: state variables, algebraic variables, derivatives, internal values, and parameters
3. `_gather_functions()` **collects all mathematical functions** from the involved modules and **resolves variable references** through the alias system
 - Coupled variables get properly linked
 - If one module's output feeds another's input, the alias mapping ensures they reference the same underlying variables

Physical Value und Unit Handling:

- Module is trying to solve a solved problem
- Make only **lightweight class for PhysicalVariable using Pint's capabilities**
- **Split unit logic into a back-end wrapper around Pint**, reducing class overlap
- **Unit correctness is not enforced during model evaluation**

Coupled Block:

- Has significant common functionality with **Module class**, could **benefit from a common base class**
 - evaluate differential, algebraic, internal, and common functions
 - solve ode step, solve algebraic step
 - Improves consistency, maintainability, extensibility, polymorphism, and reduces code
- **Complex variable management**: string-based variable naming and alias resolution
- same as for module `_evaluate_internal()` and `_evaluate_common()` can **require many iterations**
- **Hard coded solver methods**
- **Complex level assignment** using integer and floating points for level and sub-level

Module:

- **Constructor** is doing several tasks **violating Single Responsibility Principle** (manage meta data, initialize solver settings, initialize data containers, load module definitions, apply user overrides, conversion of units of physical values, setting up execution state)
- **`_load_description_file()`**: violating Single Responsibility Principle, loading, parsing, validating and storing of the module definition
- **`_resolve_io()`** does **not detect duplicate physical values** as inputs/outputs that might override
- **`_get_decorated_functions()`**: could **perform a dependency analysis and dependency validation**, and could suggest already an execution order by detecting cycles
- **`_evaluate_internal()`**: could perform a **static dependency analysis before to get the internal execution order**, would remove the need for iterate through the equations until all dependencies could be resolved
- **`initialize()`**: breaks if an algebraic loop cannot be solved, the **method cannot distinguish between resolvable (direct feed-through) and true circular (algebraic loop) dependencies**
- **`step()`**: does not handle discrete state variable with **explicit event handling**

Simulation:

- **Discrete State and Event Handling**: no discrete state support, no event detection, fixed time stepping, no mode switching, floating-point time representation is not appropriate for ensuring simultaneity of events
- **External Model Integration**: no external interface, only Python modules are supported, no co-simulation, cannot integrate with external solvers, no data exchange protocols
- **Heterogeneous Time Integration**: No adaptive time stepping regardless of dynamics, no rollback capability
- **Flat simulation Hierarchy**: no abstraction or nesting of modules/subsystems

OpenModelica

- Free, open source, highly portable Python based interactive session handler for Modelica scripting
- Provides the modeler with components for creating a complete Modelica modeling, compilation and simulation environment based on the latest OpenModelica library standard available
- OMPython is designed to combine both the solving strategy and model building
- OMPython is not a standalone package, it depends upon the OpenModelica installation
- OMPython is implemented in Python and depends on ZeroMQ - high performance asynchronous messaging library

```
from OMPython import ModelicaSystem

# Load the Modelica model
model = ModelicaSystem("ClassicModel.mo", "ClassicModel")

# Define simulation options
stopTime = 140

model.setSimulationOptions([f"stopTime={stopTime}",
                           "stepSize=0.01",
                           "tolerance=1e-6",])

# Simulate the model
print("Simulating the model...")
model.simulate()

# Get the simulation results
t = model.getSolutions('time').flatten()
derx = model.getSolutions('der(x)').flatten()
x = model.getSolutions('x').flatten()
dery = model.getSolutions('der(y)').flatten()
y = model.getSolutions('y').flatten()
```

OpenSim

- OpenSim scripting with Python is available via pre-built Conda package